

1 What are the fundamental differences between the original ASP.NET and ASP.NET Core?

ASP.NET

- Works only on Windows
- Heavy and slower
- Uses System.Web

ASP.NET Core

- Works on Windows, Linux, Mac
- Faster and lightweight
- More flexible and modern

2 What does it mean for an API to be "Stateless?"

- server **does NOT remember anything** about previous requests
- Each request is **independent**

3 Break down the anatomy of an HTTP Request URL?

<https://www.myapp.com/Api/data?q=abc123&date=2004/04/04>

https => schema

www.myapp.com => host

Api/data? => path

q=abc123 => query string

4 What are the primary HTTP Methods (Verbs) and their standard uses?

- Get => get the data
- Post => create new data

- Put => update data
- Delete => delete data

5 What is the role of Program.cs in an ASP.NET Core application?

- Start the app
- Add services
- Add configuration

6 Why is Swagger/OpenAPI typically enabled only in the "Development" environment?

- Test all end points
- Security risks
- Code =>

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

7 What is the core concept of "Dependency Injection" (DI)?

- System gives you object without dependency

8 Explain the three Service Lifetimes in ASP.NET Core DI?

- transits
- scoped
- singleton

9 Why is it a "Best Practice" to register services against an Interface rather than a concrete Class?

	BEFORE (TIGHT COUPLING)	AFTER (CONSTRUCTOR INJECTION + INTERFACE)
new	Inside the class	Outside
Coupling	To a concrete class	To an interface (abstraction)
Testing	Hard — real database	Easy — mock/fake
Maintainability	Change in 20 places	Change in one place
Flexibility	Can't swap the implementation	Swap with one line

10 What are the "Launch Profiles" found in launchSettings.json?

- Defines how app runs

11 How does JSON facilitate data exchange in APIs?

- Used to send/receive data between client & server
- Easy to read
- Lightweight
- Works with all languages

Question 1 - What is Dependency?

إيه المشاكل في الكود ده؟

- Hard to test
- Poor maintainability
- Low flexibility
- Poor scalability

Question 2 - Tight Coupling Problem

What's the difference between A and B? Which one is better?

- Scenario B is the best
- Use interface
- Flexible in implementation
- Easy to test

Question 3 - Constructor Injection

What happens when a request is received?

Request

- ↓ server receive the request and go to controller

Controller

- ↓ DI

ProductService

- ↓

IRepository → SqlRepository make instance

- ↓

Database

- ↓

Response

Question 4 - DI Container Registration

What is the output for each registration?

- { "AreSameInstance": false }
- { "AreSameInstance": true }
- { "AreSameInstance": true }

Question 5 - Multiple Registrations

Which implementation will Controller A receive?

How many services will be injected into Controller B?

1- Receive MailgunEmailService

2-

- SmtplibEmailService
- SendGridEmailService
- MailgunEmailService